

AI-powered bioinformatic analysis using R, #8

Strings, Iteration & Functional Programming

Wei-Hua Chen (CC BY-NC 4.0)

HUST, China

2026-07-17

Table of contents I

- 1 String Basics
- 2 Regular Expressions (Regex)
- 3 Regex Tasks in stringr
- 4 Iteration: for Loops
- 5 The apply Family
- 6 purrr: Modern Iteration
- 7 Writing Custom Functions
- 8 Parallel Computing (Brief)
- 9 Summary

Section 1

String Basics

What Are Strings?

Strings are sequences of characters. In bioinformatics, strings are everywhere:

- DNA/RNA sequences: "ATCGGCGCG . . . "
- Gene names: "TP53", "BRCA1"
- File paths, sample IDs, annotations

Some examples

```
library(tidyverse)
library(stringr)

string1 <- "This is a string"
string2 <- 'If I want to include a "quote" inside a string, I use single quotes'
```

```
# Multiline string
(string3 <- "a multiline
string")
```

```
[1] "a multiline\nstring"
```

```
writeLines(string3) # see the actual content
```

```
a multiline
string
```

Special Characters

```
# Double and single quotes
(double_quote <- "\"")
```

```
[1] "\""
```

```
(single_quote <- "'")
```

```
[1] "'"
```

```
# Unicode
(x <- "\u00b5") # µ (micro sign)
```

```
[1] "µ"
```

```
# Backslash — note the difference!
(y <- "\\")
```

```
[1] "\\"
```

```
writeLines(y)
```

```
\
```



String Length

```
# Base R  
nchar(c("a", "R for data science", NA))
```

```
[1]  1 18 NA
```

```
# stringr  
str_length(c("a", "R for data science", NA))
```

```
[1]  1 18 NA
```

String Concatenation

```
# Base R — paste()
paste("a", "b", "c", sep = "")
```

```
[1] "abc"
```

```
# stringr — str_c()
str_c("a", "b", "c")
```

```
[1] "abc"
```

```
# Vectorized
paste(c("a", "b", "c"), 2, sep = "")
```

```
[1] "a2" "b2" "c2"
```

```
str_c(c("a", "b", "c"), 2)
```

```
[1] "a2" "b2" "c2"
```



String Comparison

```
# Direct comparison (lexicographic)
```

```
"A" > "abc"
```

```
[1] FALSE
```

```
library(pracma)
```

```
strcmp("chen", "chenweihua")    # case-sensitive
```

```
[1] FALSE
```

```
strcmpi("chen", "CHEN")          # case-insensitive
```

```
[1] TRUE
```

Case Conversion & Other Basics

```
toupper(letters[1:10])
```

```
[1] "A" "B" "C" "D" "E" "F" "G" "H" "I" "J"
```

```
tolower(LETTERS[1:5])
```

```
[1] "a" "b" "c" "d" "e"
```

```
library(stringi)  
stri_reverse("ABC")
```

```
[1] "CBA"
```

Tip: stringi functions start with `stri_`, stringr functions start with `str_`.

Section 2

Regular Expressions (Regex)

What is Regex?

A sequence of characters that define a search pattern.

```
# [ab] means "a or b"
```

```
c("abc", "chen", "liu", "blah") %>% str_subset("[ab]")
```

```
[1] "abc" "blah"
```

```
# \d+ means "one or more digits"
```

```
c("a1334bc", "ch13e_45n", "liu", "b100ah") %>% str_extract("\\d+")
```

```
[1] "1334" "13" NA "00"
```

Useful tools: regex101.com, regexr.com

Four Tasks of Regex

- ① **Match patterns** — what characters?
- ② **Match rules** — which character classes?
- ③ **Position rules** — where in the string?
- ④ **Quantity rules** — how many repetitions?

1. Character Classes & Groups

Pattern	Meaning
.	any character
[abc]	a, b, or c
[^abc]	anything except a, b, c
[a-z]	any lowercase letter
[A-Z]	any uppercase letter
[0-9]	any digit
(ab\\ cd)	"ab" or "cd"

1. Special Character Classes

Shorthand	Meaning
<code>\d</code>	digit [0-9]
<code>\D</code>	non-digit
<code>\w</code>	word character [A-Za-z0-9_]
<code>\W</code>	non-word
<code>\s</code>	whitespace (space, tab, newline)
<code>\S</code>	non-whitespace

```
"abc_123_?$$^" %>% str_detect("\\s+") # has whitespace?
```

```
[1] FALSE
```

```
"abc_123_?$$^" %>% str_detect("\\d+") # has digits?
```

```
[1] TRUE
```

```
"abc_123_?$$^" %>% str_detect("\\w+") # has word chars?
```

```
[1] TRUE
```



2. Position Rules — Anchors

Anchor	Meaning
^	start of string
\$	end of string
\b	word boundary

```
# Ends with "wei"
c("chen wei hua", "chen wei", "chen") %>% str_subset("wei$")
```

```
[1] "chen wei"
```

```
# Word ending in "wei"
c("chen wei hua", "chen wei", "chen") %>% str_subset("wei\\b")
```

```
[1] "chen wei hua" "chen wei"
```



3. Quantity Rules — Quantifiers

Quantifier	Meaning
+	1 or more
*	0 or more
?	0 or 1
{n}	exactly n
{n,}	n or more
{n,m}	between n and m

```
"1234abc" %>% str_extract("\\d+")      # all digits
```

```
[1] "1234"
```

```
"1234abc" %>% str_extract("\\d{3}")    # exactly 3 digits
```

```
[1] "123"
```

```
"1234abc" %>% str_extract("\\d{5,6}")  # 5-6 digits → NA
```

```
[1] NA
```



Section 3

Regex Tasks in stringr

1. Detect Patterns

```
# Base R  
grep("\\d+", c("123", "abc", "wei555hua"))
```

```
[1] 1 3
```

```
grep("\\d+", c("123", "abc", "wei555hua"))
```

```
[1] TRUE FALSE TRUE
```

```
# stringr  
c("123", "abc", "wei555hua") %>% str_detect("\\d+")
```

```
[1] TRUE FALSE TRUE
```

1. Count Matches

```
x <- c("why", "video", "cross", "extra", "deal", "authority")  
str_detect(x, "[aeiou]")    # contains vowel?
```

```
[1] FALSE TRUE TRUE TRUE TRUE TRUE
```

```
str_count(x, "[aeiou]")    # how many vowels?
```

```
[1] 0 3 1 2 2 4
```

2. Locate Patterns

```
# Base R
regexpr("\\d+", c("123", "abc", "wei555hua"))
```

```
[1] 1 -1 4
attr(,"match.length")
[1] 3 -1 3
attr(,"index.type")
[1] "chars"
attr(,"useBytes")
[1] TRUE
```

```
# stringr
c("123", "abc", "wei555hua") %>% str_locate("\\d+")
```

```
      start end
[1,]      1  3
[2,]     NA NA
[3,]      4  6
```



3. Extract Patterns

```
c("123", "abc", "wei555hua") %>% str_extract("\\d+")
```

```
[1] "123" NA      "555"
```

```
c("123", "abc", "wei555hua") %>% str_match("\\d+")
```

```
      [,1]  
[1,] "123"  
[2,] NA  
[3,] "555"
```

str_extract vs. str_match

```
x
```

```
[1] "why"      "video"    "cross"    "extra"    "deal"     "authority"
```

```
str_extract(x, "(.)[aeiou](.)") # extract match
```

```
[1] NA      "vid" "ros" NA      "dea" "aut"
```

```
str_match(x, "(.)[aeiou](.)") # extract groups around vowel
```

```
      [,1] [,2] [,3]
[1,] NA   NA   NA
[2,] "vid" "v"  "d"
[3,] "ros" "r"  "s"
[4,] NA   NA   NA
[5,] "dea" "d"  "a"
[6,] "aut" "a"  "t"
```

str_match() returns capture groups (...) as separate columns!



Extract All Occurrences

```
x
```

```
[1] "why"      "video"    "cross"    "extra"    "deal"     "authority"
```

```
str_extract_all(x, "[aeiou]+")
```

```
[[1]]
```

```
character(0)
```

```
[[2]]
```

```
[1] "i"  "eo"
```

```
[[3]]
```

```
[1] "o"
```

```
[[4]]
```

```
[1] "e" "a"
```

```
[[5]]
```

```
[1] "ea"
```

```
[[6]]
```

```
[1] "a" "a" "a" "a" "a" "a"
```



4. Replace Patterns

```
str_replace(c("123", "abc", "wei555hua"), "\\d+", "###")
```

```
[1] "###"      "abc"      "wei###hua"
```

```
str_replace_all("123_abc_456_789", "\\d+", "###")
```

```
[1] "###_abc_###_###"
```

5. Split by Pattern

```
str_split(x, "")
```

```
[[1]]
```

```
[1] "w" "h" "y"
```

```
[[2]]
```

```
[1] "v" "i" "d" "e" "o"
```

```
[[3]]
```

```
[1] "c" "r" "o" "s" "s"
```

```
[[4]]
```

```
[1] "e" "x" "t" "r" "a"
```

```
[[5]]
```

```
[1] "d" "e" "a" "l"
```

```
[[6]]
```

```
[1] "a" "u" "t" "h" "o" "r" "i" "t" "y"
```

Advanced Example: Chromosome Coordinates

```
dat <- tibble(chrom = readLines(textConnection(
  "chr11:69464719-69502928
chr7:55075808-55093954
chr8:128739772-128762863
chr3:169389459-169490555
chr17:37848534-37877201
chr19:30306758-30316875
chr1:150496857-150678056
chr12:69183279-69260755
chr11:77610143-77641464
chr8:38191804-38260814
chr12:58135797-58156509"))))

# Split into chr, start, end
dat$chrom %>% str_split('[:-]', simplify = TRUE)
```

	[,1]	[,2]	[,3]
[1,]	"chr11"	"69464719"	"69502928"
[2,]	"chr7"	"55075808"	"55093954"
[3,]	"chr8"	"128739772"	"128762863"
[4,]	"chr3"	"169389459"	"169490555"



Alternative: `tidyr::extract()`

```
library(tidyr)
extract(dat, chrom, into = c('chr', 'chrStart', 'chrEnd'),
        '([^-:~+]):([^-~+])-(.*)', convert = TRUE)
```

A tibble: 11 x 3

	chr	chrStart	chrEnd
	<chr>	<int>	<int>
1	chr11	69464719	69502928
2	chr7	55075808	55093954
3	chr8	128739772	128762863
4	chr3	169389459	169490555
5	chr17	37848534	37877201
6	chr19	30306758	30316875
7	chr1	150496857	150678056
8	chr12	69183279	69260755
9	chr11	77610143	77641464
10	chr8	38191804	38260814
11	chr12	58135797	58156509

Section 4

Iteration: for Loops

for Loop Basics

```
library(tidyverse)

df <- tibble(a = rnorm(100), b = rnorm(100), c = rnorm(100), d = rnorm(100))

# Compute row means — pre-allocate output!
res1 <- vector("double", nrow(df))
for (row_idx in 1:nrow(df)) {
  res1[row_idx] <- mean(as.numeric(df[row_idx, ]))
}

# Compute column means
res2 <- vector("double", ncol(df))
for (col_idx in 1:ncol(df)) {
  res2[col_idx] <- mean(df[[col_idx]])
}
```

Golden rule: Always **pre-allocate** the output vector! `vector("double", N)`



Vectorized Alternatives to for Loops

```
# Much faster than for loops
rowMeans(df)
```

```
[1] -0.95395346 -0.85307261  0.86963946 -0.25495517 -0.72860579  0.25371026
[7] -0.21775855  0.22952998  0.34951603 -0.12288553 -0.10130772  0.10723994
[13] -0.28391085  0.72959904  1.26396979  0.68223165  0.31740311 -0.07208439
[19]  0.48971660  0.01324365 -0.06689520  0.96455423  0.18212489  0.59706749
[25]  0.55183554 -0.92827651  0.21757147  0.09336534 -0.47114826  0.04675708
[31]  0.40183062 -0.12409047  0.21873777 -0.06369765 -0.06301541 -0.15966365
[37] -0.36265713  0.14629349 -0.01951988 -0.44149400  0.14090380  0.93052102
[43] -0.12156128 -0.51393629  0.28236922 -0.90910869  0.05399394  0.22057566
[49]  0.66234303 -0.21233809 -0.46338635 -0.51990005  0.29707884 -0.50924422
[55] -0.50106558  0.68923748  0.30130196  0.21884789  0.64340505 -1.38629940
[61]  0.09506409  0.27014504 -0.35496462 -0.01420900  0.25859583 -0.07650945
[67] -0.40760434 -0.18994491  0.03345365  0.43716885 -1.05936945  0.53817647
[73]  0.63364902  0.86878245  0.27580778 -0.48278125  0.42856507 -0.46543276
[79] -0.20832413  0.89482536  0.02810922 -0.11324258  0.21206008  0.05329016
[85] -0.42741735  0.14009559 -0.25415149 -0.59412015 -0.04136146  0.58843107
[91] -0.01387251 -0.38543992 -0.73716361  0.29674914 -0.29013099 -0.28546398
[97]  0.90637101 -0.67653128  0.08036515  0.06424800
```

```
colMeans(df)
```



Section 5

The apply Family

apply() — Row/Column Operations

`apply(X, MARGIN, FUN, ...)`

- **MARGIN = 1:** rows
- **MARGIN = 2:** columns

```
df %>% apply(1, median)    # row medians
```

```
[1] -1.3862571255 -0.8069501777  1.2911832173 -0.4466948219 -0.4879723790
[6]  0.1387968113  0.1305933623  0.3830912098  0.8547182325  0.0713415508
[11] -0.0159188164 -0.0642735984 -0.3199376212  0.8364687140  1.3456049236
[16]  0.3396711010  0.3949410122 -0.2524377764  0.6842119509  0.1848589062
[21] -0.0335114318  0.7655016532  0.0675881122  0.6434644960  0.3200854959
[26] -0.7332615770  0.0028713091  0.2283535649 -0.4869684890 -0.2819346955
[31]  0.2036721665  0.1096215047  0.2200871487 -0.3492498886  0.1905818867
[36]  0.0578435742 -0.2441390665  0.2552976040  0.1781755101 -0.4313988306
[41]  0.1216350362  0.4624569075 -0.0871899146 -0.8286703101  0.0377551357
[46] -1.0797331969  0.0962559280  0.0630137534  0.4543854473  0.0004106074
[51] -0.5250911031 -0.4288733039  0.4094778980 -0.1964503023 -0.4369756270
[56]  0.4633173289 -0.0099593050  0.2612472676  0.6240167046 -1.4363795052
[61] -0.1182775052  0.5090619675 -0.4667605296  0.0932996006  0.3085781126
[66] -0.0125409967  0.0210209642 -0.0487093821 -0.1114659642  0.3997759817
```



apply() with Custom Functions

```
df %>% apply(2, function(x) {
  return(c(n = length(x), mean = mean(x), median = median(x)))
})
```

	a	b	c	d
n	100.00000000	100.00000000	1.000000e+02	100.00000000
mean	-0.01339209	-0.03677498	7.206275e-03	0.1136248
median	-0.04656347	0.05070729	8.260712e-02	0.1996932

Note: Most row operations can be replaced by `dplyr`.

tapply() — Grouped Operations

```
tapply(X, INDEX, FUN)
```

Group **X** by **INDEX**, apply **FUN** to each group.

```
library(magrittr)
# mpg by cylinder
mtcars %$% tapply(mpg, cyl, mean)
```

```
      4      6      8
26.66364 19.74286 15.10000
```

tapply vs. dplyr

```
# tapply
mtcars %$% tapply(mpg, cyl, mean)
```

```
      4      6      8
26.66364 19.74286 15.10000
```

```
# dplyr — clearer, more readable
mtcars %>% group_by(cyl) %>% summarise(mean = mean(mpg))
```

```
# A tibble: 3 x 2
  cyl  mean
<dbl> <dbl>
1     4  26.7
2     6  19.7
3     8  15.1
```

Both tapply and dplyr operate on rows!

lapply() and sapply()

Column/list operations — iterate over each element:

- **vector**: one element at a time
- **data.frame** / **tibble** / **matrix**: one column at a time
- **list**: one member at a time

```
df %>% lapply(mean)    # returns list
```

```
$a  
[1] -0.01339209
```

```
$b  
[1] -0.03677498
```

```
$c  
[1] 0.007206275
```

```
$d  
[1] 0.1136248
```

sapply() — Type-stable

```
df %>% sapply(mean)    # returns vector
```

a	b	c	d
-0.013392093	-0.036774978	0.007206275	0.113624834

lapply/sapply with Custom Functions

```
list(a = 1:10, b = letters[1:5], c = LETTERS[1:8]) %>%  
  sapply(function(x) { length(x) })
```

```
  a  b  c  
10 5  8
```

Key: lapply/sapply = column-wise operations, equivalent to `apply(x, 2, FUN)`.

Section 6

purrr: Modern Iteration

What is purrr?

Part of the **tidyverse**, provides consistent, type-stable iteration functions.

`map()` = `lapply()` replacement:

- Iterates over each column (tibble) or slot (list)
- Applies FUN
- Returns results as a list

map() — Column/Slot Operations

```
df %>% map(summary)
```

\$a

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
-2.14055	-0.67923	-0.04656	-0.01339	0.78236	1.96369

\$b

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
-3.07996	-0.52834	0.05071	-0.03677	0.55647	1.76897

\$c

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
-2.375992	-0.525231	0.082607	0.007206	0.691221	2.158374

\$d

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
-2.8153	-0.5030	0.1997	0.1136	0.5920	2.5257

Typed map Functions (supply replacement)

Function	Returns
<code>map_lgl()</code>	logical vector
<code>map_int()</code>	integer vector
<code>map_dbl()</code>	double vector
<code>map_chr()</code>	character vector
<code>map_df()</code>	data frame (row-binding)

```
df %>% map_dbl(mean)
```

```
      a      b      c      d
-0.013392093 -0.036774978 0.007206275 0.113624834
```



Advanced map: split + cor.test

```
# For each cylinder group, correlate mpg and wt
mtcars %>%
  split(.$cyl) %>%
  map(~ cor.test(.$wt, .$mpg)) %>%
  map_dbl(~ .$estimate)
```

```
           4           6           8
-0.7131848 -0.6815498 -0.6503580
```

The code logic:

- split the data frame by cyl
- for each group, run `cor.test(wt, mpg)`
- extract the estimate from the result

The code above is equivalent to:

```
# For each cylinder group, correlate mpg and wt
mtcars %>%
  group_by(cyl) %>%
  summarise(cor = cor.test(wt, mpg)$estimate)
```

```
# A tibble: 3 x 2
```

	cyl	cor
	<dbl>	<dbl>
1	4	-0.713
2	6	-0.682
3	8	-0.650

The ~ (tilde) Syntax

Short for `function(x):`

Full form

```
map(function(df) { cor.test(df$wt, df$mpg) })
```

Shorthand — `~` replaces `function`, `.` replaces the argument

```
map(~ cor.test(.$wt, .$mpg))
```

map2() — Two-Input Mapping

```
mu <- list(5, 10, -3)
sigma <- list(1, 5, 10)
map2(mu, sigma, rnorm, n = 5) %>% str()
```

List of 3

```
$ : num [1:5] 5.14 4.62 4.5 5.67 5.33
$ : num [1:5] 11.36 18.83 2.15 9.75 10.73
$ : num [1:5] -29.14 -1.53 -6.4 -4.03 -7.64
```

What is the output?

- generate 5 random numbers for each pair of `mu` (as mean) and `sigma` (as standard deviation)

pmap() — Multi-Input Mapping

```
params <- tribble(
  ~mean, ~sd, ~n,
    5,    1,  1,
  10,    5,  3,
  -3,   10,  5
)
params %>% pmap(rnorm)
```

```
[[1]]
```

```
[1] 5.18633
```

```
[[2]]
```

```
[1] 16.373495 11.985756  7.359677
```

```
[[3]]
```

```
[1] -2.985803  4.073842 -9.395030 -4.526504  1.011967
```

reduce() — Combine Elements

```
# Full join multiple tables
dfs <- list(
  age = tibble(name = "John", age = 30),
  sex = tibble(name = c("John", "Mary"), sex = c("M", "F")),
  trt = tibble(name = "Mary", treatment = "A")
)
dfs %>% reduce(full_join)
```

```
# A tibble: 2 x 4
  name      age sex  treatment
  <chr> <dbl> <chr> <chr>
1 John      30 M      <NA>
2 Mary      NA F       A
```

reduce() for Set Operations

```
vs <- list(  
  c(1, 3, 5, 6, 10),  
  c(1, 2, 3, 7, 8, 10),  
  c(1, 2, 3, 4, 8, 9, 10)  
)  
vs %>% reduce(intersect)    # common elements
```

```
[1] 1 3 10
```

accumulate() — See Intermediate Steps

```
(x <- sample(10))
```

```
[1] 4 5 1 8 7 2 10 9 3 6
```

```
x %>% accumulate(`+`) # cumulative sum, step by step
```

```
[1] 4 9 10 18 25 27 37 46 49 55
```

keep() and discard()

```
# Keep factor columns
iris %>% keep(is.factor)
```

	Species
1	setosa
2	setosa
3	setosa
4	setosa
5	setosa
6	setosa
7	setosa
8	setosa
9	setosa
10	setosa
11	setosa
12	setosa
13	setosa
14	setosa
15	setosa
16	setosa
17	setosa

discard()

```
# Discard factor columns
iris %>% discard(is.factor)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width
1	5.1	3.5	1.4	0.2
2	4.9	3.0	1.4	0.2
3	4.7	3.2	1.3	0.2
4	4.6	3.1	1.5	0.2
5	5.0	3.6	1.4	0.2
6	5.4	3.9	1.7	0.4
7	4.6	3.4	1.4	0.3
8	5.0	3.4	1.5	0.2
9	4.4	2.9	1.4	0.2
10	4.9	3.1	1.5	0.1
11	5.4	3.7	1.5	0.2
12	4.8	3.4	1.6	0.2
13	4.8	3.0	1.4	0.1
14	4.3	3.0	1.1	0.1
15	5.8	4.0	1.2	0.2
16	5.7	4.4	1.5	0.4
17	5.4	3.9	1.3	0.4
18	5.1	3.5	1.4	0.2



Section 7

Writing Custom Functions

Function Syntax

```
# Basic structure
my_function <- function(arg1, arg2 = default_value) {
  # body
  return(result)
}

# Example
calc_cv <- function(x) {

  ## if no return value is specified, the last expression is returned
  sd(x, na.rm = TRUE) / mean(x, na.rm = TRUE) * 100
}

calc_cv(mtcars$mpg)
```

```
[1] 29.99881
```



Defensive Programming

Always validate inputs:

```
safe_mean <- function(x) {  
  stopifnot(is.numeric(x))      # validate input  
  if (length(x) == 0) return(NA) # handle edge case  
  mean(x, na.rm = TRUE)  
}  
  
# safe_mean(c("a", "b")) # would error with clear message  
safe_mean(c(1, 2, 3, NA))
```

```
[1] 2
```

Section 8

Parallel Computing (Brief)

Why Parallel Computing?

- Modern computers have multiple CPU cores
- Many bioinformatics tasks are “embarrassingly parallel”
- Speed up by running tasks simultaneously on multiple cores

Three Steps of Parallel Computing

- ① **Split** tasks and distribute to workers
- ② **Compute** independently on each worker
- ③ **Collect** and combine results

Key Packages

- `parallel`: detect CPU cores, `parLapply()`
- `doParallel`: register parallel backend
- `foreach`: provides `%do%` (sequential) and `%dopar%` (parallel)
- `iterators`: split data for parallel tasks

Simple Example

```
library(doParallel)
library(foreach)
library(iterators)
```

```
# Detect CPUs
(cpus <- parallel::detectCores())
```

```
[1] 10
```

```
# Register parallel backend
registerDoParallel(cpus - 1)  # leave 1 core free
```

```
# Parallel loop
```

```
data(iris)
```

```
x <- iris[which(iris[, 5] != "setosa"), c(1, 5)]
```

```
trials <- 10000
```

```
r <- foreach(icount(trials), .combine = rbind) %dopar% {
  ind <- sample(100, 100, replace = TRUE)
  result1 <- glm(x[ind, 2] ~ x[ind, 1], family = binomial(logit))
  coefficients(result1)
}
```

Sequential vs. Parallel: Time Comparison

Let's compare running time of sequential vs. parallel for a realistic bioinformatics task — performing many t-tests on simulated gene expression data:

```
library(doParallel)
library(foreach)

# Simulate: 5000 genes × 20 samples (10 case, 10 control)
set.seed(123)
n_genes <- 5000
expr_data <- matrix(rnorm(n_genes * 20, mean = 10, sd = 2), nrow = n_genes)
colnames(expr_data) <- c(paste0("Case_", 1:10), paste0("Ctrl_", 1:10))
group <- rep(c("Case", "Ctrl"), each = 10)

# --- Sequential ---
time_seq <- system.time({
  res_seq <- foreach(i = 1:n_genes, .combine = c) %do% {
    t.test(expr_data[i, group == "Case"],
           expr_data[i, group == "Ctrl"])$p.value
  }
})
```

Sequential vs. Parallel: Time Comparison, cont.

```
# Register parallel backend
registerDoParallel(parallel::detectCores() - 1)

# --- Parallel ---
time_par <- system.time({
  res_par <- foreach(i = 1:n_genes, .combine = c) %dopar% {
    t.test(expr_data[i, group == "Case"],
           expr_data[i, group == "Ctrl"])$p.value
  }
})

stopImplicitCluster()
```

Sequential vs. Parallel: Time Comparison, cont.

```
# Compare timing
rbind(Sequential = time_seq[1:3], Parallel = time_par[1:3])

      user.self sys.self elapsed
Sequential    0.456    0.003    0.460
Parallel      0.168    0.027    0.263

# Speedup
cat(sprintf("\nSpeedup: %.1fx faster with parallel computing\n",
           time_seq["elapsed"] / time_par["elapsed"]))
```

Speedup: 1.7x faster with parallel computing

Key takeaway: For independent, repetitive computations (e.g., per-gene statistics, bootstrapping, permutation tests), parallel computing can dramatically reduce runtime.

foreach Parameters

- `.combine`: how to combine results (`'c'`, `'rbind'`, `'cbind'`, or a custom function)
- `.packages`: packages to load on each worker (e.g., `c("tidyverse")`)

```
# Example with .packages
res <- foreach(df = iter(mtcars %>% split(.$cyl)),
               .combine = 'rbind',
               .packages = c("ggplot2")) %dopar% {
  p <- ggplot(df, aes(x = wt, y = mpg)) + geom_point()
  return(p)
}

# ALWAYS close the cluster!
stopImplicitCluster()
```

furrr: purrr + future

A simpler approach — combine purrr with future:

```
library(furrr)
plan(multisession, workers = 4) # use 4 cores

# Replace map() with future_map()
result <- mtcars %>%
  split(.$cyl) %>%
  future_map(~ cor.test(.$wt, .$mpg))
```

Section 9

Summary

What We Covered

Topic	Key Functions
String basics	<code>str_length</code> , <code>str_c</code> , <code>toupper</code> , <code>tolower</code>
Regex	<code>str_detect</code> , <code>str_count</code> , <code>str_locate</code> , <code>str_extract</code> , <code>str_match</code> , <code>str_replace</code> , <code>str_split</code>
for loops	pre-allocation, <code>vector("double", N)</code>
apply family	<code>apply()</code> , <code>tapply()</code> , <code>lapply()</code> , <code>sapply()</code>
purrr	<code>map()</code> , <code>map_dbl()</code> , <code>map2()</code> , <code>pmap()</code> , <code>reduce()</code> , <code>accumulate()</code> , <code>keep()</code> , <code>discard()</code>
Custom functions	<code>function()</code> , <code>stopifnot()</code> , input validation
Parallel computing	<code>foreach + %dopar%</code> , <code>furrr::future_map()</code>

Next Talk

Talk 09: Statistical Analysis & Machine Learning Basics

Homework

Complete Homework_AI_powered_R/talk08-homework.qmd